



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

05 Divide and Conquer

CS216 Algorithm Design and Analysis (H)

Instructor: Shan Chen

chens3@sustech.edu.cn



Divide-and-Conquer Paradigm

- **Divide and Conquer 分治:**

- **Divide** up problem into several subproblems (of the same kind).
- **Conquer** (solve) each subproblem **recursively**.
- **Combine** solutions to subproblems into solution to original problem.

- **Most common usage:**

- **Divide** problem of size n into **two** subproblems of size $n/2$.
- **Conquer** (solve) two subproblems **recursively**.
- **Combine** two solutions into solution to original problem.

- **Common time complexity:**

- Brute Force: $\Theta(n^2)$
- Divide and Conquer: $O(n \log n)$ ← $T(n) = 2T(n/2) + O(n)$

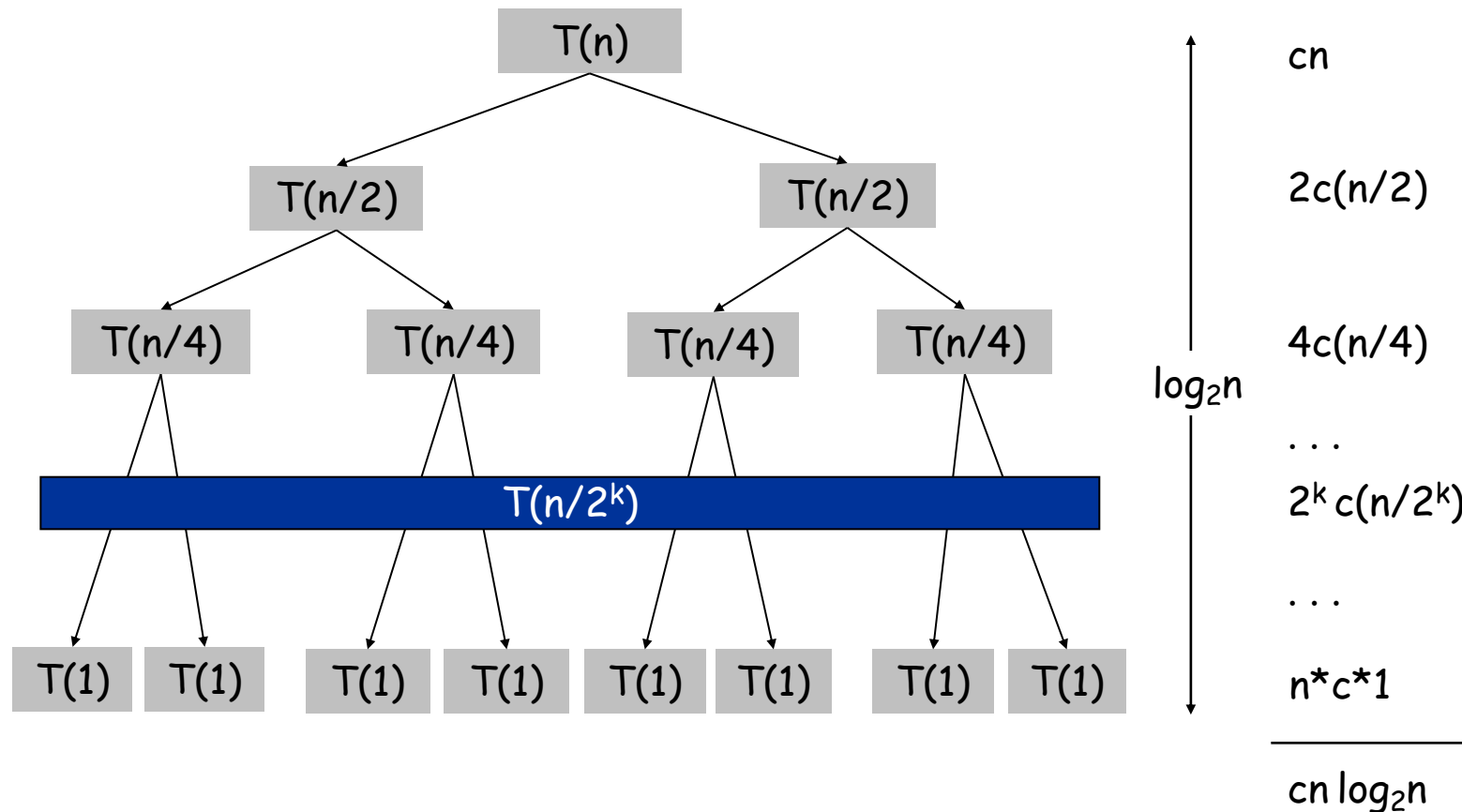
↖ e.g., combine in $O(n)$ time

凡治众如治寡
孙子兵法
分数是也



Divide-and-Conquer Recurrences

- **Example:** $T(n) = 2T(n/2) + O(n)$ with $T(1) = \Theta(1)$





Divide-and-Conquer Recurrences

- **Divide-and-Conquer recurrences.** $T(n) = aT(n/b) + f(n)$ with $T(1) = \Theta(1)$
- **Master Theorem.** Let $a \geq 1$, $b \geq 2$, $c \geq 0$ and suppose $T(n)$ is a function on the non-negative integers that satisfies the recurrence

$$T(n) = aT(n/b) + \Theta(n^c)$$

with $T(1) = \Theta(1)$, where n/b means either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then,

- Case 1: If $c > \log_b a$, then $T(n) = \Theta(n^c)$.
- Case 2: If $c = \log_b a$, then $T(n) = \Theta(n^c \log n)$.
- Case 3: If $c < \log_b a$, then $T(n) = \Theta(n^{\log_b a})$.



1. Counting Inversions



Counting Inversions

- **Match song preferences:**

- Every person ranks n songs.
- Goal: Music site consults database to find people with **similar tastes**.

- **Similarity metric.** Number of **inversions** between two rankings.

- My rank: $1, 2, \dots, n$
- Your rank: $a_1, a_2, \dots, a_n$
- Songs i and j are **inverted** if $i < j$ but $a_i > a_j$.

Songs					
	A	B	C	D	E
Me	1	2	3	4	5
You	1	3	4	2	5

inversions
3-2, 4-2

- **Brute Force.** Check all $\Theta(n^2)$ pairs.

- **Q.** Can we count inversions faster?



Recall: Merge Sort

- **Merge Sort:**

- **Divide** array into **two halves**.
- **Recursively** sort each half.
- **Merge** two halves to make sorted whole.

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	L	G	O	R		I	T	H	M	S
---	---	---	---	---	--	---	---	---	---	---

A	G	L	O	R		H	I	M	S	T
---	---	---	---	---	--	---	---	---	---	---

A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

divide $O(n)$

sort $2T(n/2)$

merge $O(n)$

total $O(n \log n)$



Jon von Neumann (1945)



Counting Inversions: Divide and Conquer

- **Divide and Conquer:**

- **Divide:** separate list into two pieces.
- **Conquer:** recursively count inversions in each half.
- **Combine:** count inversions where a_i and a_j are in **different halves** and return sum of three quantities.

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

Divide: $O(n)$

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

Conquer: $2T(n/2)$

5 blue-blue inversions

8 green-green inversions

5-4, 5-2, 4-2, 8-2, 10-2 6-3, 9-3, 9-7, 12-3, 12-7, 12-11, 11-3, 11-7

9 blue-green inversions

5-3, 4-3, 8-6, 8-3, 8-7, 10-6, 10-9, 10-3, 10-7

Count: $O(n)$?

how to count blue-green inversions ?

Total = 5 + 8 + 9 = 22.



Counting Inversions: Divide and Conquer

- **Divide and Conquer:**

- **Divide:** separate list into two pieces.
- **Conquer:** recursively count inversions in each half. easy to count if both sorted
- **Combine:** count inversions where a_i and a_j are in **different halves** and return sum of three quantities.

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

Divide: $O(n)$

two
pointers

1	2	4	5	8	10	3	6	7	9	11	12
						4	2	2	1	0	0

Conquer: $2T(n/2)$

blue-green inversions: $4 + 2 + 2 + 1 + 0 + 0 = 9$

Count: $O(n)$

Merge-and-Count: $O(n)$

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

Merge: $O(n)$



Counting Inversions: $O(n \log n)$ Algorithm

- **Sort-and-Count algorithm:**

```
Sort-and-Count(L) {  
    if (list L has one element) return (0, L)  
  
    Divide the list into two halves A and B  
    ( $r_A$ , A)  $\leftarrow$  Sort-and-Count(A)  
    ( $r_B$ , B)  $\leftarrow$  Sort-and-Count(B)  
    ( $r_{AB}$ , L)  $\leftarrow$  Merge-and-Count(A, B)  
  
    return ( $r_A + r_B + r_{AB}$ , L)  
}
```

- **Merge-and-Count sub-algorithm:**

- Input: A and B are sorted (by Sort-and-Count).
- Output: L is sorted.



2. Closest Pair of Points



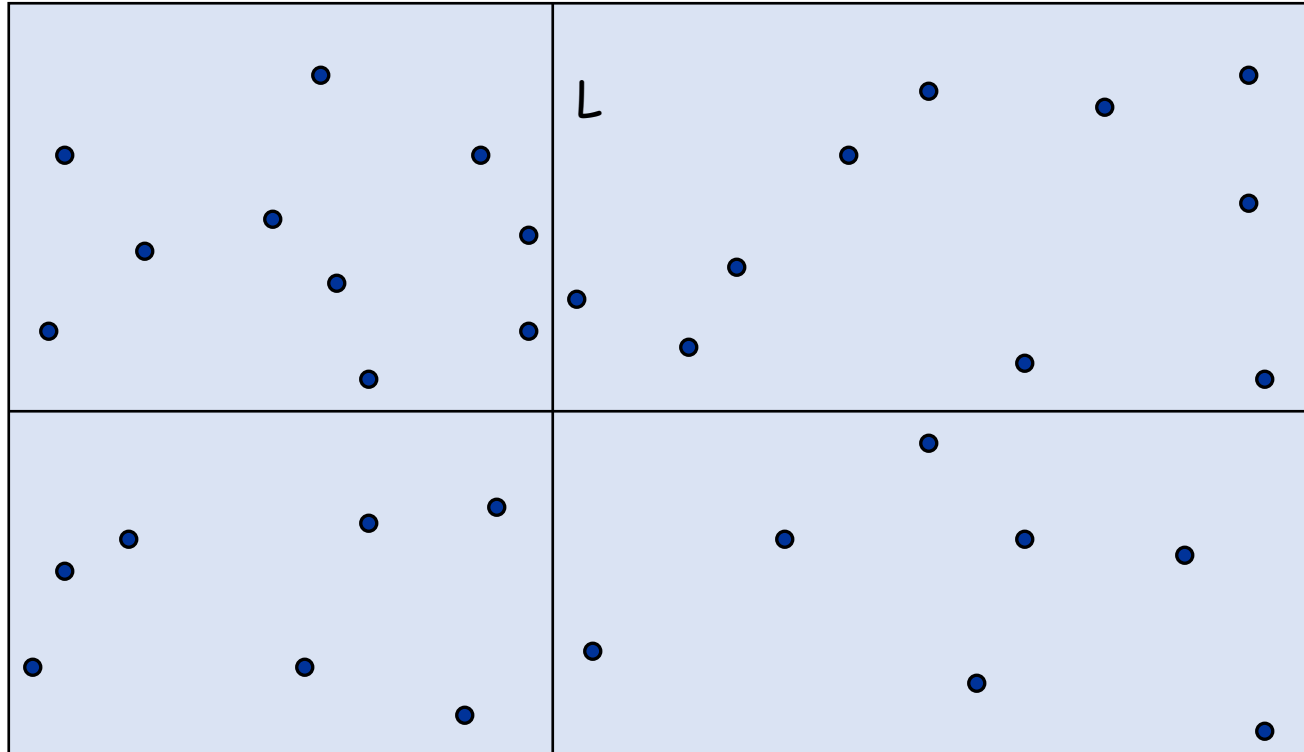
Closest Pair of Points

- **Closest Pair 最近点对 Problem.** Given n points in the plane, find a pair with smallest Euclidean distance between them.
 - **Fundamental geometric primitive:**
 - Example applications: graphics, computer vision, geographic information systems, molecular modeling, air traffic control, etc.
 - Special case of **nearest neighbor, Euclidean MST, Voronoi diagram**, etc.
- fast closest pair algorithm inspires fast algorithms for these problems
- **Brute Force.** Check all pairs with $\Theta(n^2)$ distance calculations.
 - **1-D version.** Easy $O(n \log n)$ algorithm if points are **on a line**.
 - **2-D version.** Can we beat $\Theta(n^2)$? Can it be solved in $O(n \log n)$ time too?



Closest Pair of Points: A First Attempt

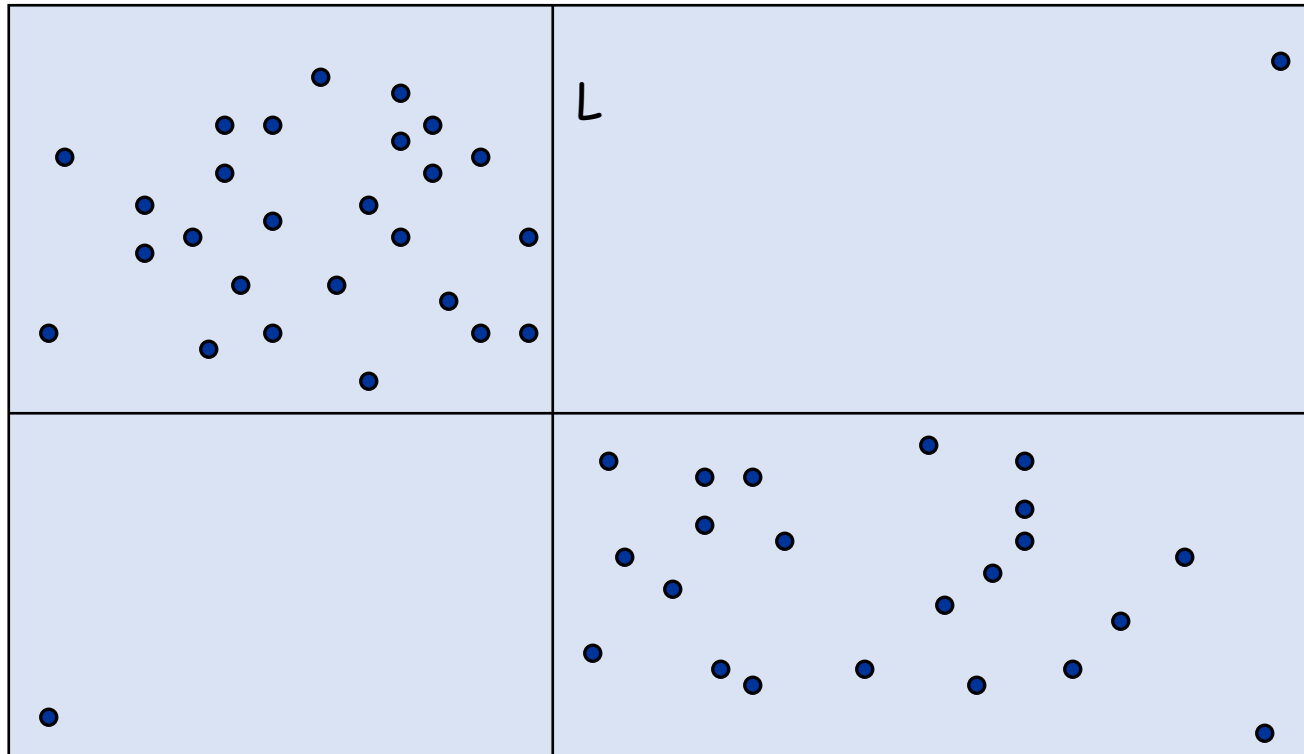
- **Divide.** Sub-divide region into **4** quadrants.





Closest Pair of Points: A First Attempt

- **Divide.** Sub-divide region into **4** quadrants.
- **Obstacle.** Impossible to ensure **$n/4$** points in each quadrant.

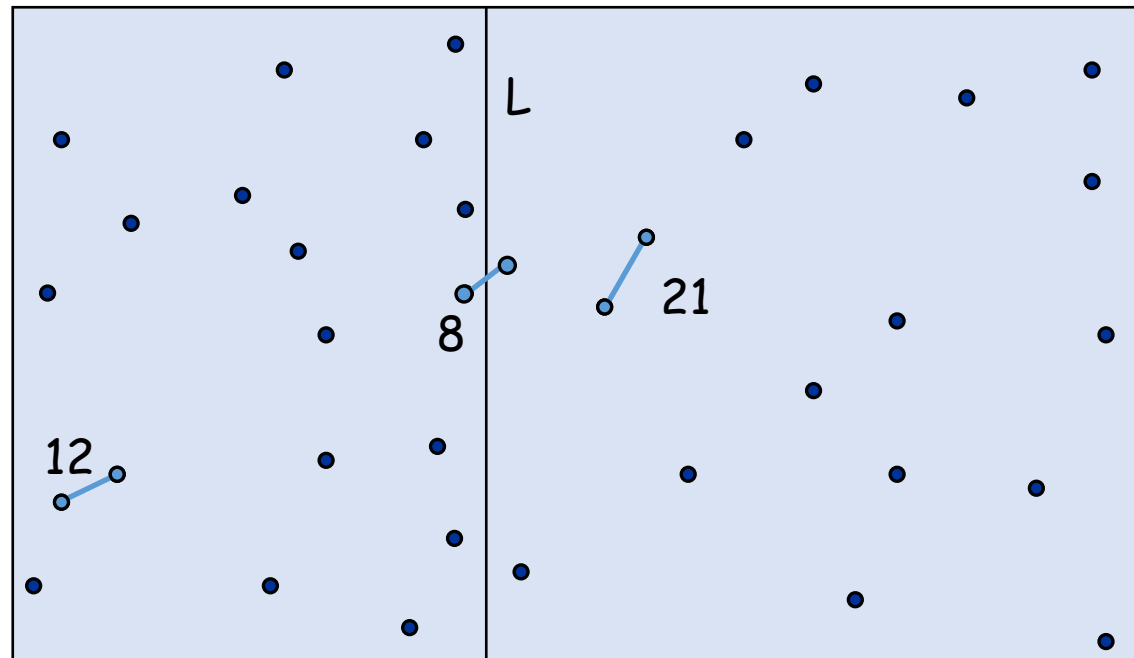




Closest Pair of Points: Divide and Conquer

- **Divide and Conquer:**

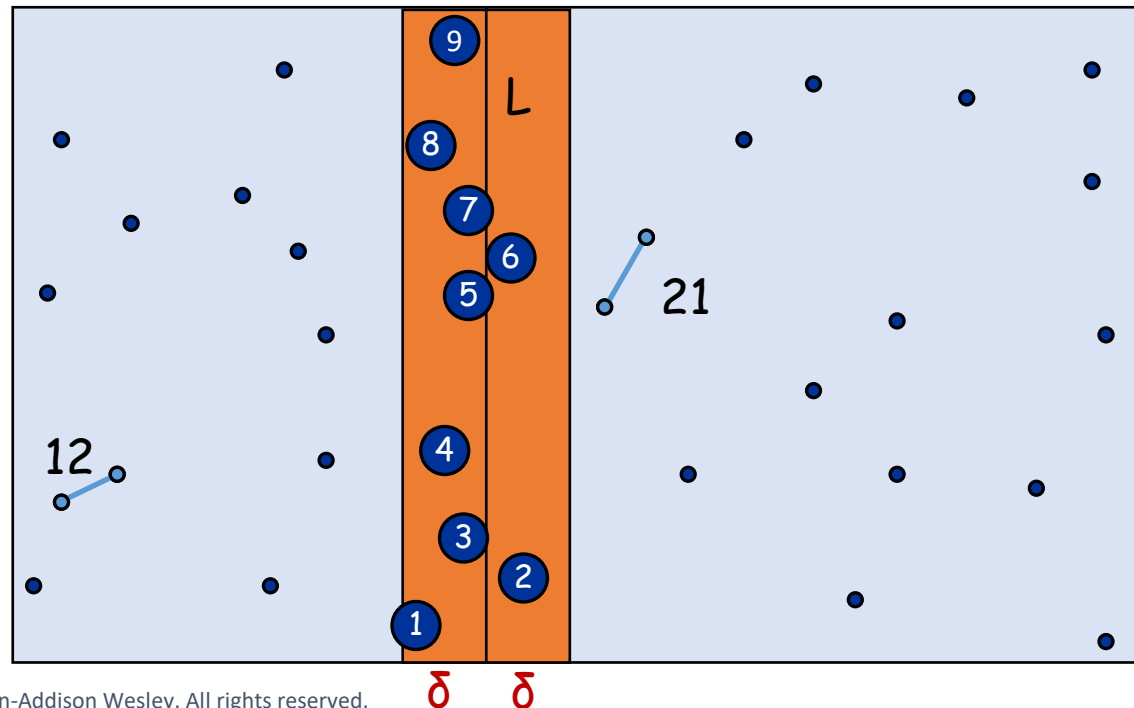
- **Divide:** draw vertical line L so that roughly $n/2$ points lie in each side.
 - **Conquer:** find closest pair in each side **recursively**.
 - **Combine:** find closest pair **across two sides** and return best of **3** solutions.
- ← can we find it in $\Theta(n)$ time?





Closest Pair of Points: Divide and Conquer

- **Combine step.** Find closest pair with **one point in each side**.
 - **Observation:** suffices to consider only points **within δ** of line L , where δ is the distance of closest pair with **both points in one side**.
known from recursion
 - Sort points in 2δ -strip by their y -coordinate.
 - Check distances of only those points in the sorted list **within 7 positions!**
why this works?



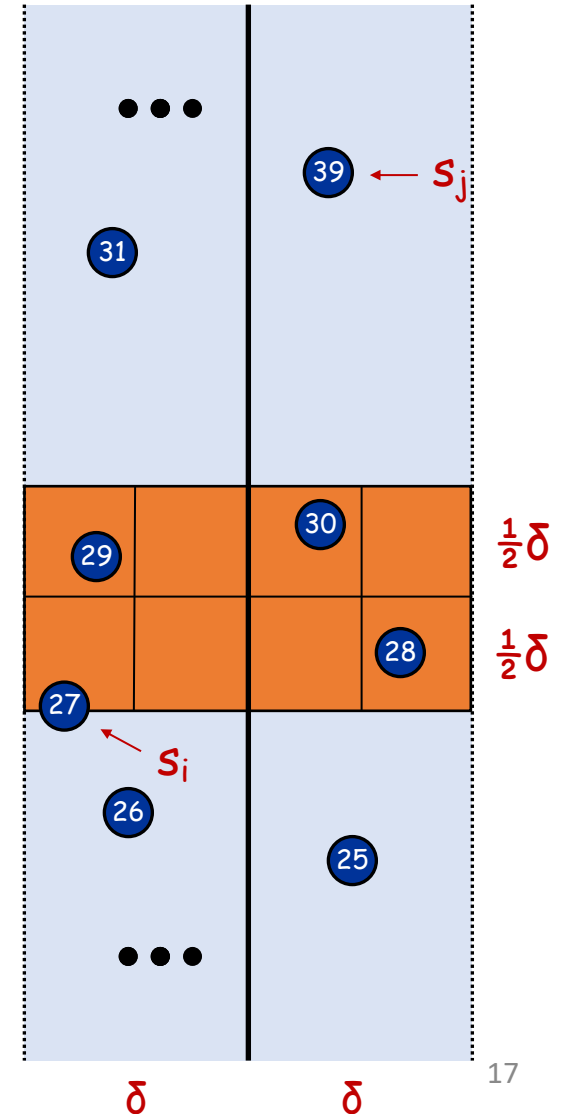
$$\delta = \min(12, 21)$$



Closest Pair of Points: Divide and Conquer

- **Claim.** Let s_i be the point in the 2δ -strip with the i -th smallest y -coordinate. If $|i - j| > 7$, then the distance between s_i and s_j is at least δ .
- **Pf. (direct proof)**
 - Consider the 2δ -by- δ rectangle R in strip whose min y -coordinate is y -coordinate of s_i .
 - Distance between s_i and any point s_j above R is $\geq \delta$.
 - Subdivide R into 8 squares.
 - At most 1 point per square. ← because square diameter $< \delta$
 - At most 7 other points can be in R . ▀

constant can be improved with more refined argument





Closest Pair of Points: $O(n \log n)$ Algorithm

- **Divide-and-Conquer algorithm:** (sort by x-axis and y-axis beforehand)

```
Closest-Pair( $p_1, \dots, p_n$ ) {  
  Compute vertical line  $L$  such that half the points  
  are on each side of the line and Partition the points.  
  
   $\delta_1 \leftarrow \text{Closest-Pair}(\text{left half})$   
   $\delta_2 \leftarrow \text{Closest-Pair}(\text{right half})$   
   $\delta \leftarrow \min(\delta_1, \delta_2)$   
  
  Delete all points further than  $\delta$  from line  $L$ .  
  
  Sort remaining points by y-coordinate.  
  
  Scan points in y-order and compare distance between  
  each point and next 7 neighbors. If any of these  
  distances is less than  $\delta$ , update  $\delta$ .  
  
  return  $\delta$   
}
```

$O(n)$ use x-sorted list

$2T(n/2)$

$O(n)$

~~$O(n \log n)$~~ use y-sorted list

$O(n)$



Closest Pair of Points: Closing Remarks

- [Rabin 1976] There exists a **randomized** algorithm that finds the closest pair of points in the plane with **expected** running time $O(n)$.
- **Remark.** There are ingenious **divide-and-conquer** algorithms for **core geometric problems**.
 - **3D or higher dimensions** test limits of our ingenuity.

problem	brute	clever
closest pair	$O(n^2)$	$O(n \log n)$
farthest pair	$O(n^2)$	$O(n \log n)$
convex hull	$O(n^2)$	$O(n \log n)$
Delaunay/Voronoi	$O(n^4)$	$O(n \log n)$
Euclidean MST	$O(n^2)$	$O(n \log n)$

running time to solve a 2D problem with n points



Announcement

- **Lab 6 will be released today and the deadline is Apr 15.**



3. Integer Multiplication



Integer Addition and Subtraction

- **Addition.** Given two n -bit integers a and b , compute $a + b$.
- **Subtraction.** Given two n -bit integers a and b , compute $a - b$.
- **Grade-school addition/subtraction algorithm.** $\Theta(n)$ bit operations.
 - Subtraction can be transformed to addition (using **two's complement** 补码).

	1	1	1	1	1	1	0	1	
		1	1	0	1	0	1	0	1
+	0	1	1	1	1	1	1	0	1
	1	0	1	0	1	0	0	1	0

- **Remark.** Grade-school addition and subtraction algorithms are **optimal**.



- [illegible]



Integer Multiplication: A First Attempt

- **Divide and Conquer:** (multiply two n -bit integers x and y)
 - Divide x and y into low- and high-order bits.
 - Recursively multiply **four** $n/2$ -bit integers: ac, bd, bc, ad
 - Add and shift to obtain result.

- **Example.** $n = 8, m = \lceil n/2 \rceil = 4$.

$$x = \underbrace{1000}_{a} \underbrace{1101}_{b} \quad y = \underbrace{1110}_{c} \underbrace{0001}_{d}$$

$$xy = (2^m a + b)(2^m c + d) = 2^{2m} \underbrace{ac}_{1} + 2^m (\underbrace{bc}_{2} + \underbrace{ad}_{3}) + \underbrace{bd}_{4}$$

- **Time complexity.** $\Theta(n^2)$ $\longleftarrow T(n) = 4T(n/2) + O(n)$



- $$\begin{array}{c}
 x = \underbrace{10001101}_a \quad \underbrace{11000001}_c \quad y = \underbrace{111000001}_c \quad \underbrace{1}_d \\
 \\
 xy = (2^m a + b)(2^m c + d) = 2^{2m} \underbrace{ac}_1 + 2^m \underbrace{(ac + bd - (a - b)(c - d))}_{\substack{1 \quad 3 \quad 2}} + \underbrace{bd}_3
 \end{array}$$

- **Time complexity.** $\Theta(n^{\log_2 3}) = \Theta(n^{1.585}) \leftarrow T(n) = 3T(n/2) + O(n)$



Karatsuba Algorithm

- **Karatsuba algorithm:** (for integers longer than one 32/64-bit word)

```
Karatsuba-Multiply(x, y, n) {  
  if (n = 1) return x * y  
  else  
    m ← ⌊ n / 2 ⌋  
    a ← ⌊ x / 2m ⌋; b = x mod 2m  
    c ← ⌊ y / 2m ⌋; d = y mod 2m  
    e ← Karatsuba-Multiply(a, c, m)  
    f ← Karatsuba-Multiply(b, d, m)  
    g ← Karatsuba-Multiply(|a - b|, |c - d|, m)  
    Flip sign of g if needed  
  
    return 22m e + 2m(e + f - g) + f  
}
```

$O(n)$

$3T(n / 2)$

$O(n)$

- **Practice.** Faster than grade-school for integers longer than ≈ 320 bits.



Integer Multiplication: Asymptotic Complexity

year	algorithm	bit operations
12xx	grade school	$O(n^2)$
1962	Karatsuba-Ofman	$O(n^{1.585})$
1963	Toom-3, Toom-4	$O(n^{1.465})$, $O(n^{1.404})$
1966	Toom-Cook	$O(n^{1+\epsilon})$
1971	Schönhage-Strassen	$O(n \log n \cdot \log \log n)$
2007	Fürer	$n \log n 2^{O(\log^* n)}$
2019	Harvey-van der Hoeven	$O(n \log n)$
	???	$O(n)$

- **Remark.** GNU Multiple Precision Arithmetic Library (GMP) uses one of the first 5 algorithms depending on n .

used in Maple, Mathematica, gcc, cryptography...



4. Matrix Multiplication



Matrix Multiplication

- **Matrix multiplication.** Given n -by- n matrices A and B , compute $C = AB$.
- **Grade-school matrix multiplication.** $\Theta(n^3)$ arithmetic operations.
- **Block matrix multiplication:**

$$\begin{bmatrix} 152 & 158 & 164 & 170 \\ 504 & 526 & 548 & 570 \\ 856 & 894 & 932 & 970 \\ 1208 & 1262 & 1316 & 1370 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix} \times \begin{bmatrix} 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 \\ 24 & 25 & 26 & 27 \\ 28 & 29 & 30 & 31 \end{bmatrix}$$

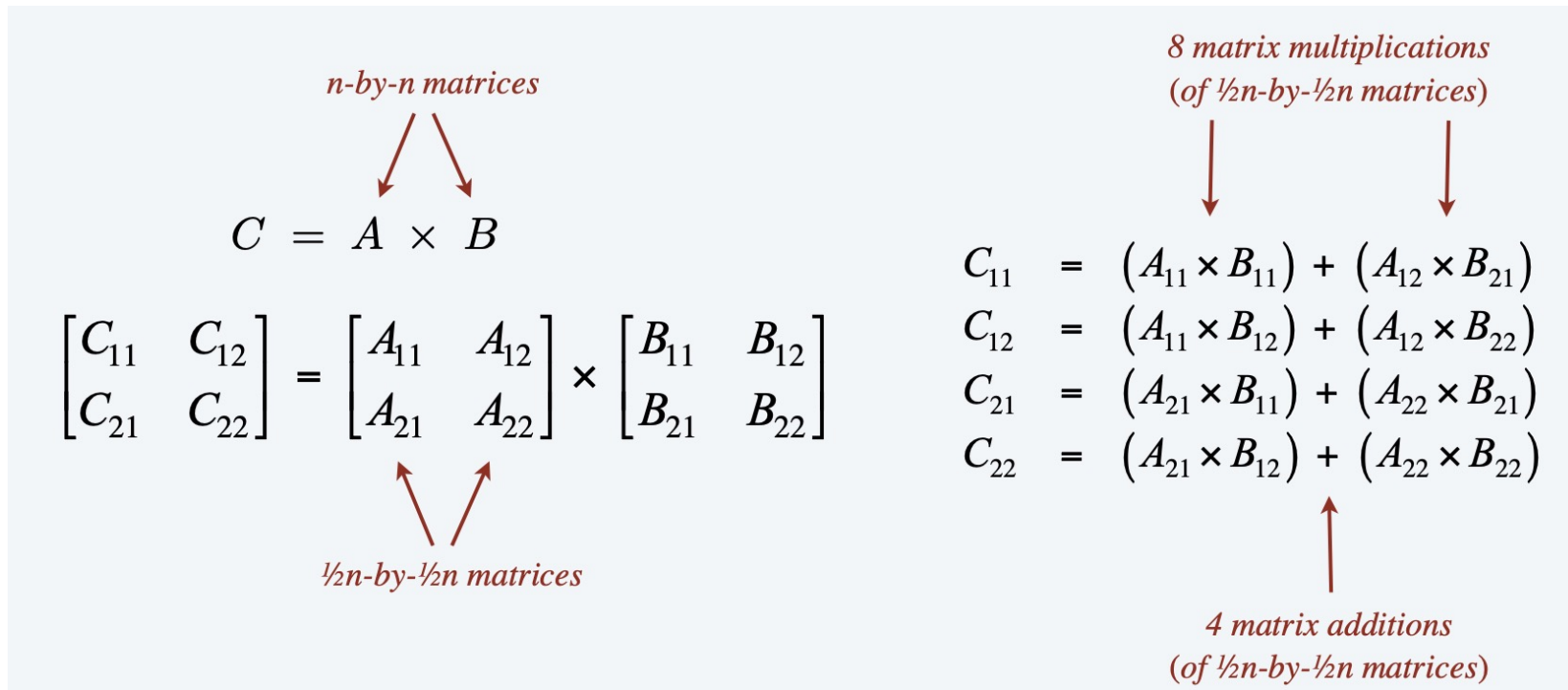
Diagram illustrating block matrix multiplication. Matrix C is partitioned into blocks C_{11} , C_{12} , C_{13} , and C_{14} . Matrix A is partitioned into blocks A_{11} , A_{12} , A_{13} , and A_{14} . Matrix B is partitioned into blocks B_{11} , B_{21} , B_{31} , and B_{41} . The equation shows $C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$.

$$C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$$



Matrix Multiplication: A First Attempt

- **Divide and Conquer:** (multiply two n -by- n matrices A and B)
 - **Divide:** partition A and B into $n/2$ -by- $n/2$ blocks.
 - **Conquer:** multiply 8 pairs of $n/2$ -by- $n/2$ matrices, recursively.
 - **Combine:** add appropriate products using 4 matrix additions.



$$T(n) = 8T(n/2) + O(n^2)$$

$$T(n) = O(n^3)$$



Matrix Multiplication: Strassen's Trick

- **Divide and Conquer:** (multiply two n -by- n matrices A and B)
 - **Divide:** partition A and B into $n/2$ -by- $n/2$ blocks.
 - **Conquer:** multiply 7 pairs of $n/2$ -by- $n/2$ matrices, recursively.
 - **Combine:** 11 matrix additions and 7 matrix subtractions.

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

scalars

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_1 + P_5 - P_3 - P_7$$

$$P_1 \leftarrow A_{11} \times (B_{12} - B_{22})$$

$$P_2 \leftarrow (A_{11} + A_{12}) \times B_{22}$$

$$P_3 \leftarrow (A_{21} + A_{22}) \times B_{11}$$

$$P_4 \leftarrow A_{22} \times (B_{21} - B_{11})$$

$$P_5 \leftarrow (A_{11} + A_{22}) \times (B_{11} + B_{22})$$

$$P_6 \leftarrow (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$P_7 \leftarrow (A_{11} - A_{21}) \times (B_{11} + B_{12})$$

$$T(n) = 7T(n/2) + O(n^2)$$

$$T(n) = O(n^{\log_2 7}) = O(n^{2.81})$$



Strassen Algorithm in Practice

- **Implementation issues:**

- Sparsity.
- Caching.
- n not a power of 2.
- Numerical stability.
- Non-square matrices.
- Storage for intermediate submatrices.
- Crossover to classical algorithm when n is “small”.
- Parallelism for multi-core and many-core architectures.



Matrix Multiplication: Asymptotic Complexity

Year	Bound	Authors
1969	2.8074	Strassen ^[1]
1978	2.796	Pan ^[9]
1979	2.780	Bini, Capovani ^[it] , Romani ^[10]
1981	2.522	Schönhage ^[11]
1981	2.517	Romani ^[12]
1981	2.496	Coppersmith, Winograd ^[13]
1986	2.479	Strassen ^[14]
1990	2.3755	Coppersmith, Winograd ^[15]
2010	2.3737	Stothers ^[16]
2012	2.3729	Williams ^{[17][18]}
2014	2.3728639	Le Gall ^[19]
2020	2.3728596	Alman, Williams ^{[20][21]}
2022	2.371866	Duan, Wu, Zhou ^[22]
2024	2.371552	Williams, Xu, Xu, and Zhou ^[23]
2024	2.371339	Alman, Duan, Williams, Xu, Xu, and Zhou ^[2]

Reference: https://en.wikipedia.org/wiki/Computational_complexity_of_matrix_multiplication

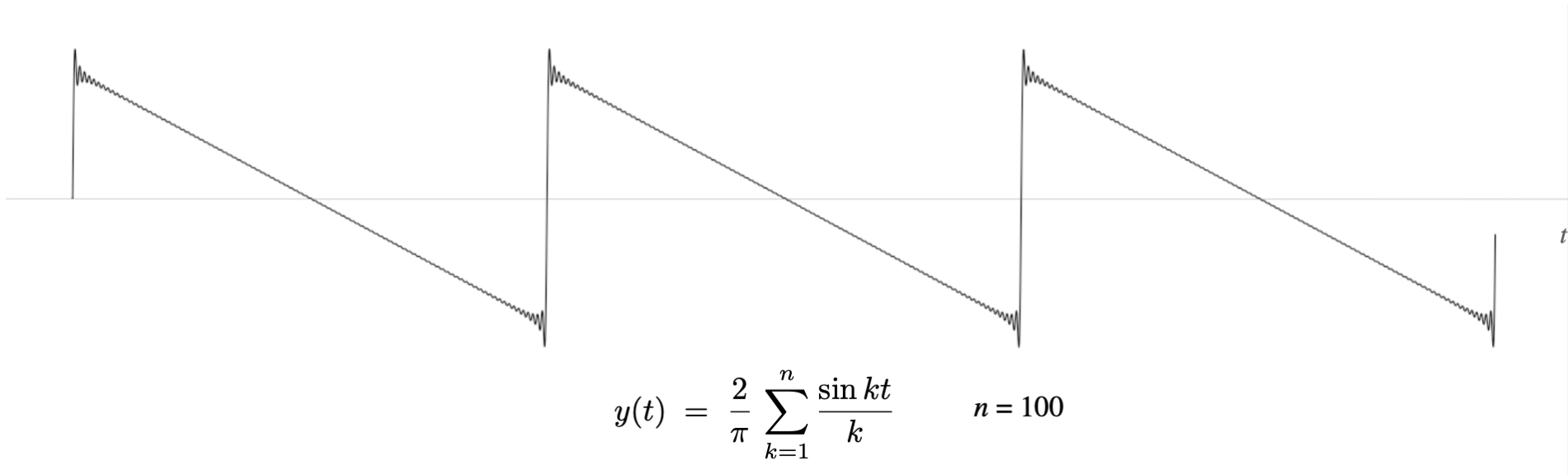


5. Convolution and FFT



Fourier Analysis and Euler's Identity

- **Fourier Theorem.** [Fourier, Dirichlet, Riemann] Any (sufficiently smooth) **periodic** function can be expressed as the **sum** of a series of **sinusoids**.



- **Euler's Formula.** $e^{ix} = \cos x + i \sin x$ **Euler's Identity.** $e^{i\pi} + 1 = 0$
 - Sum of sines and cosines = sum of complex exponentials

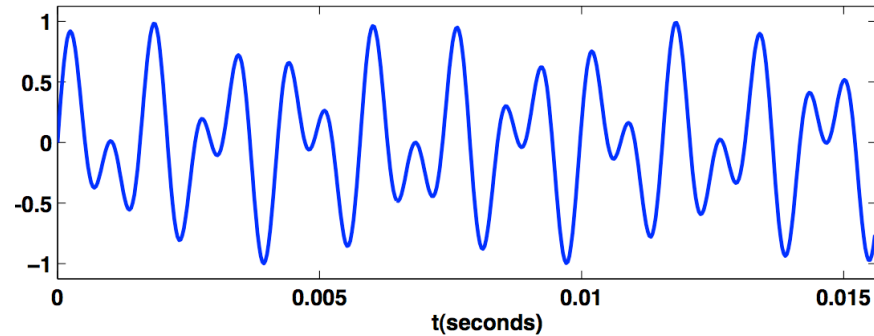


Example: Touch Tone

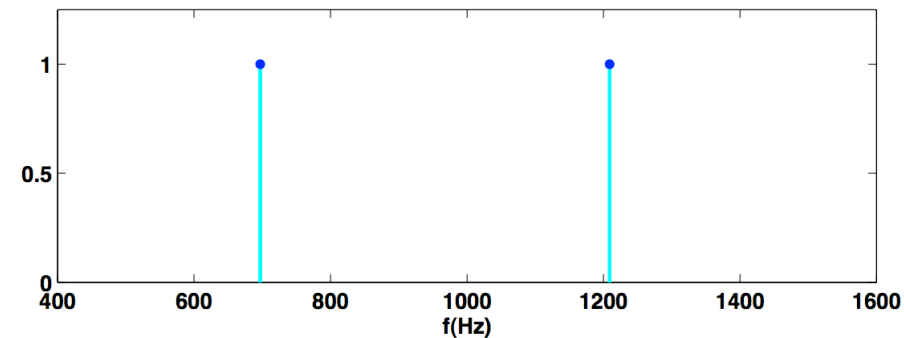
- **Signal for button 1.** $y(t) = \frac{1}{2} \sin(2\pi \cdot 697t) + \frac{1}{2} \sin(2\pi \cdot 1209t)$



- **Time domain:**



- **Frequency domain:**



Reference: Cleve Moler, Numerical Computing with MATLAB

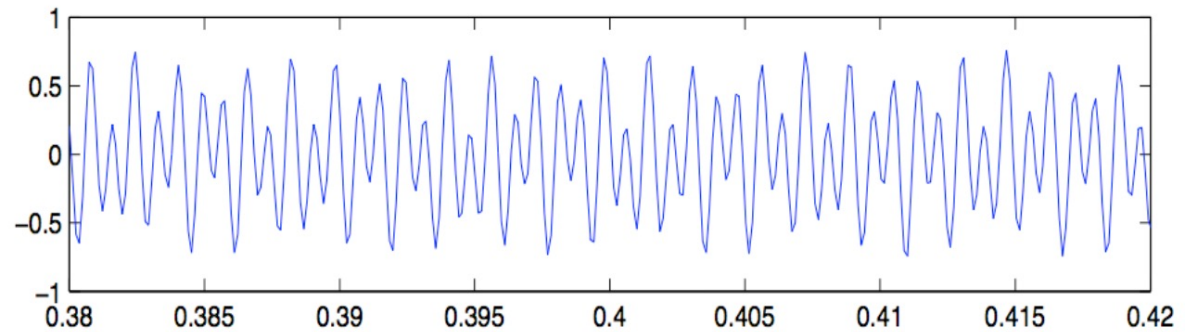


Example: Touch Tone

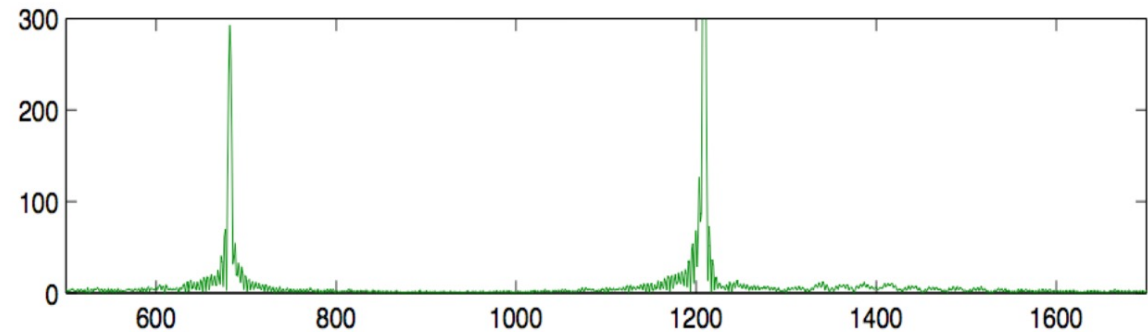
- **Signal for button 1.** $y(t) = \frac{1}{2} \sin(2\pi \cdot 697t) + \frac{1}{2} \sin(2\pi \cdot 1209t)$



- **Signal:** (sample rate ≈ 8 kHz)
 - kHz = 1000 samples/second



- **Magnitude of Discrete Fourier Transform (DFT):**



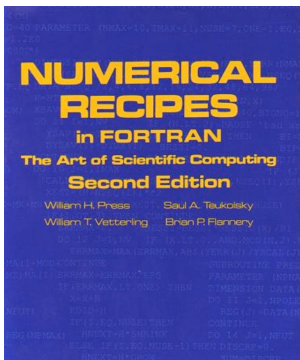
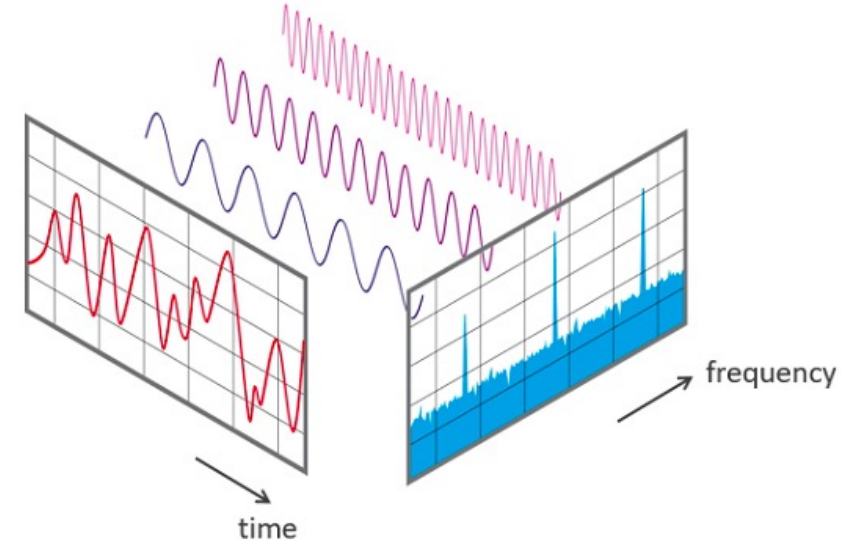
Reference: Cleve Moler, Numerical Computing with MATLAB



Fast Fourier Transform (FFT)

- **FFT.** Fast way to convert between time domain and frequency domain.
- **Alternative viewpoint.** Fast way to multiply and evaluate polynomials.

we take this viewpoint



“ If you speed up any nontrivial algorithm by a factor of a million or so the world will beat a path towards finding useful applications for it. ”



Fast Fourier Transform: Applications

- **Applications:**

- Optics, acoustics, quantum physics, telecommunications, radar, control systems, signal processing, speech recognition, data compression, image processing, seismology, mass spectrometry, integer and polynomial multiplication...
- Digital media: **DVD, JPEG, MP3, H.264**
- Medical diagnostics: **MRI, CT, PET scans, ultrasound**
- Numerical solutions to Poisson's equation.
- Shor's quantum factoring algorithm.
-

The FFT is one of the truly great computational developments of this [20th] century. It has changed the face of science and engineering so much that it is not an exaggeration to say that life as we know it would be very different without the FFT.

--- Charles van Loan



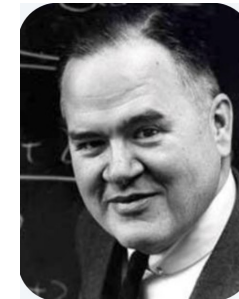
Fast Fourier Transform: Brief History

- [Gauss 1805, 1866] Analyzed periodic motion of asteroid Ceres.
- [Runge-König 1924] Laid theoretical groundwork.
- [Danielson-Lanczos 1942] Efficient algorithm, **x**-ray crystallography.
- [Cooley-Tukey 1965] Detect nuclear tests in Soviet Union and track submarines. Rediscovered and popularized **FFT**.

An Algorithm for the Machine Calculation of Complex Fourier Series

By James W. Cooley and John W. Tukey

An efficient method for the calculation of the interactions of a 2^m factorial experiment was introduced by Yates and is widely known by his name. The generalization to 3^m was given by Box et al. [1]. Good [2] generalized these methods and gave elegant algorithms for which one class of applications is the calculation of Fourier series. In their full generality, Good's methods are applicable to certain problems in which one must multiply an N -vector by an $N \times N$ matrix which can be factored into m sparse matrices, where m is proportional to $\log N$. This results in a procedure requiring a number of operations proportional to $N \log N$ rather than N^2 .



- **Note.** Importance not fully realized until emergence of **digital computers**.



Polynomials: Coefficient Representation

- **Polynomial:** (coefficient representation)

$$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} \quad B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_{n-1}x^{n-1}$$

- **Addition.** $O(n)$ arithmetic operations.

$$A(x) + B(x) = (a_0 + b_0) + (a_1 + b_1)x + \cdots + (a_{n-1} + b_{n-1})x^{n-1}$$

- **Evaluation.** $O(n)$ using **Horner's** method.

$$A(x) = a_0 + (x(a_1 + x(a_2 + \cdots + x(a_{n-2} + x(a_{n-1}))) \cdots))$$

- **Multiplication.** (linear convolution 线性卷积) $O(n^2)$ using brute force.

$$A(x) \times B(x) = \sum_{i=0}^{2n-2} c_i x^i, \text{ where } c_i = \sum_{j=0}^i a_j b_{i-j}$$



Polynomials: Point-Value Representation

- **Polynomial:** (point-value representation)

$$A(x): (x_0, y_0), \dots, (x_{n-1}, y_{n-1}) \quad B(x): (x_0, z_0), \dots, (x_{n-1}, z_{n-1})$$

- **Addition.** $O(n)$ arithmetic operations.

$$A(x) + B(x): (x_0, y_0 + z_0), \dots, (x_{n-1}, y_{n-1} + z_{n-1})$$

- **Multiplication.** $O(n)$, but represent $A(x)$ and $B(x)$ using $2n$ points.

$$A(x) \times B(x): (x_0, y_0 \times z_0), \dots, (x_{2n-1}, y_{2n-1} \times z_{2n-1})$$

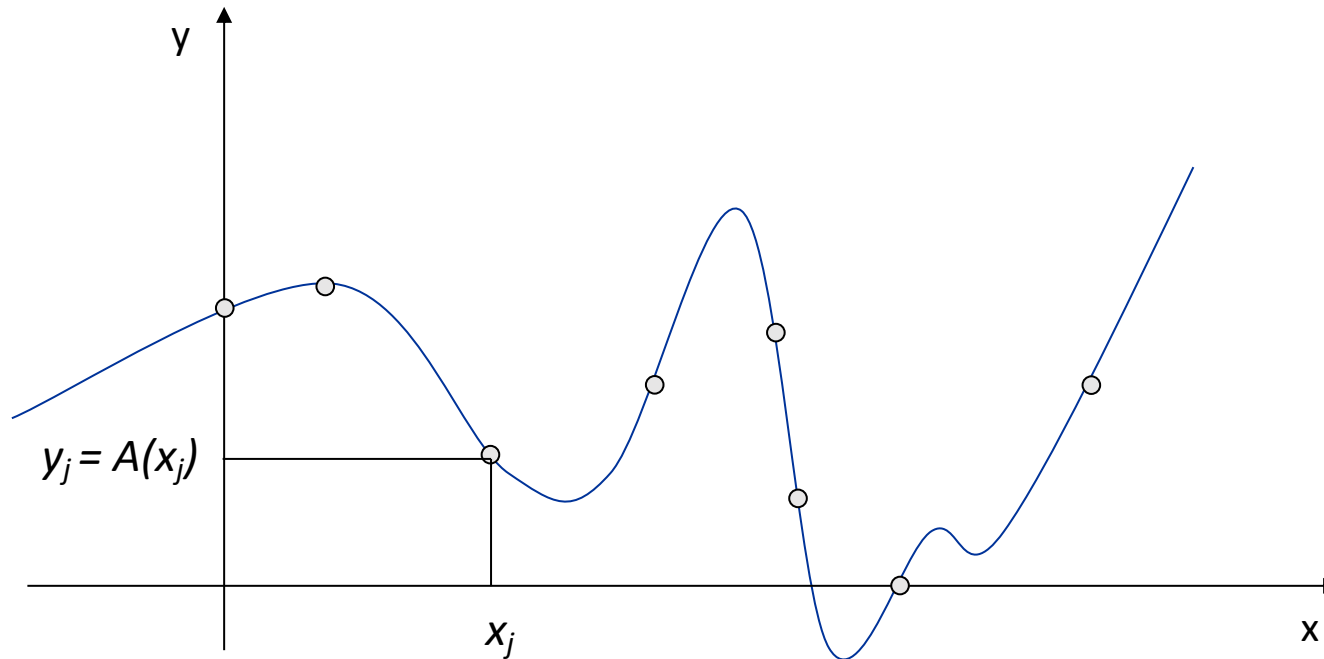
- **Evaluation.** $O(n^2)$ using Lagrange's method.

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)} \quad \leftarrow \text{not used in FFT}$$



Polynomials: Point-Value Representation

- **Fundamental Theorem of Algebra.** [Gauss, Ph.D. thesis] A degree n polynomial with complex coefficients has n complex roots.
- **Corollary.** (with Lagrange's method) A degree $n - 1$ polynomial $A(x)$ is **uniquely** specified by its evaluation at n distinct values of x .





Converting between Two Representations

- **Tradeoff.** Either fast **evaluation** or fast **multiplication**. We want both!

Representation	Multiplication	Evaluation
coefficient	$O(n^2)$	$O(n)$
point-value	$O(n)$	$O(n^2)$

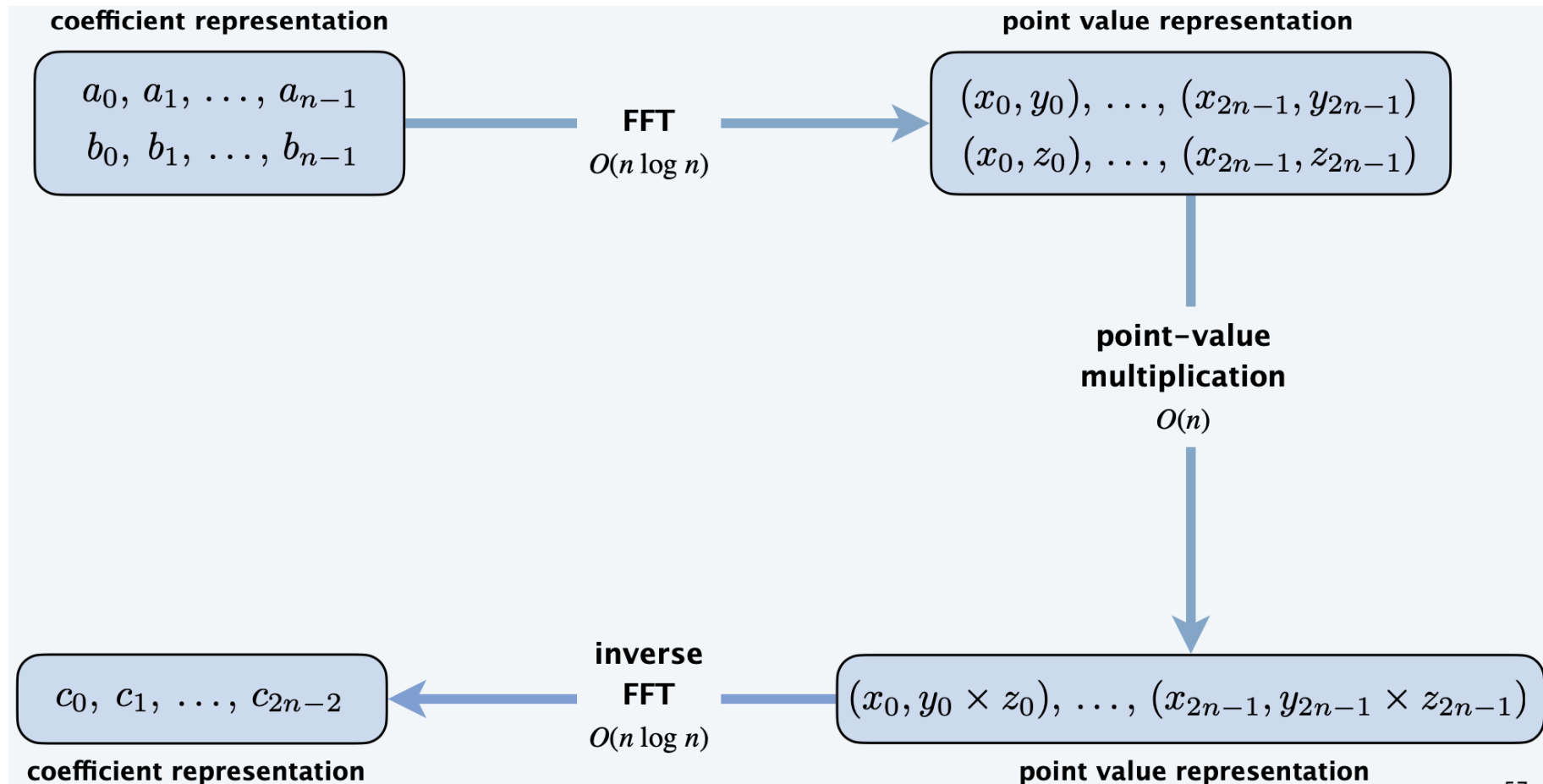
- **Goal.** **Efficient conversion** between representations $\Rightarrow$ all operations fast





Converting between Two Representations

- **Application.** Polynomial multiplication (coefficient representation).





Converting between Two Representations

- **Coefficient $\Rightarrow$ Point-Value.** Given a polynomial $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$.
 - Running time: $O(n^2)$ via matrix-vector multiplication or using Horner's n times
- **Point-Value $\Rightarrow$ Coefficient.** Given n distinct points $x_0, \dots, x_{n-1}$ and values $y_0, \dots, y_{n-1}$, find unique polynomial $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$ that has given values at given points.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

← $O(n^3)$ via Gaussian elimination
 $O(n^{2.38})$ via fast matrix multiplication

↖ Vandermonde matrix is invertible iff x_i s are distinct



Coefficient to Point-Value: Intuition

- **Coefficient $\Rightarrow$ Point-Value.** Given a polynomial $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$. $\leftarrow$ we get to choose these points!
- **Divide. [Cooley-Tukey]** Break up polynomial into **even** and **odd** degrees.

➤ $A(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + a_4 x^4 + a_5 x^5 + a_6 x^6 + a_7 x^7$

➤ $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + a_6 x^3 \qquad A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + a_7 x^3$

➤ $A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2)$

➤ $A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2)$

can also divide into **low** and **high** degrees [Sande-Tukey]

To be able to combine:
typically evaluate on the same value

- **Intuition.** Choose two points to be ± 1 .

➤ $A(1) = A_{\text{even}}(1) + 1 A_{\text{odd}}(1)$

➤ $A(-1) = A_{\text{even}}(1) - 1 A_{\text{odd}}(1)$

can evaluate polynomial of degree $n-1$ at **2** points by evaluating **2** polynomials of degree $\frac{1}{2} n - 1$ at **1** point



Coefficient to Point-Value: Intuition

- **Coefficient $\Rightarrow$ Point-Value.** Given a polynomial $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$. $\leftarrow$ we get to choose these points!
- **Divide.** [Cooley-Tukey] Break up polynomial into **even** and **odd** degrees.
 - $A(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + a_4 x^4 + a_5 x^5 + a_6 x^6 + a_7 x^7$
 - $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + a_6 x^3$ $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + a_7 x^3$
 - $A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2)$
 - $A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2)$

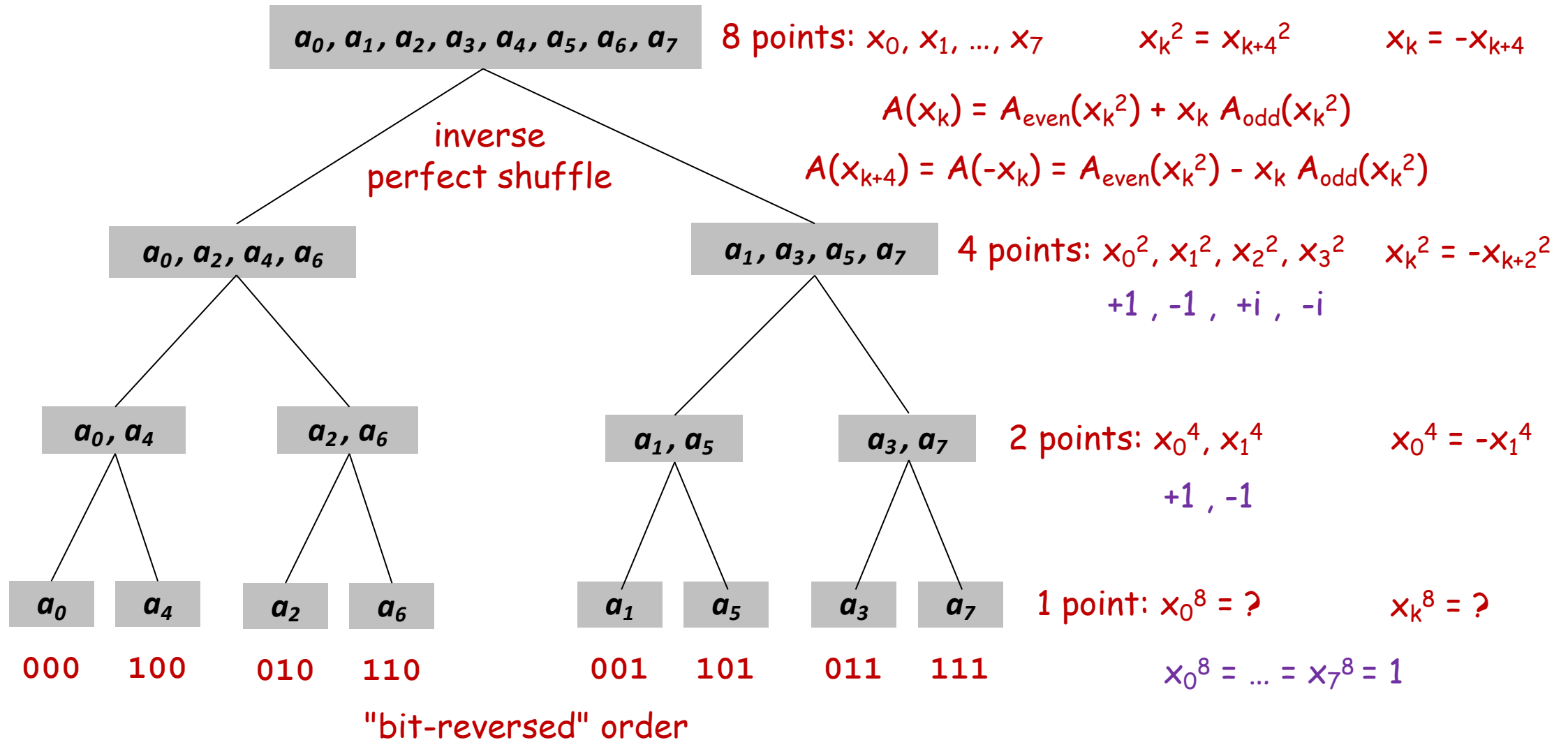
can also divide into **low** and **high** degrees [Sande-Tukey]
- **Intuition.** Choose four **complex** points to be $\pm 1, \pm i$.
 - $A(1) = A_{\text{even}}(1) + 1 A_{\text{odd}}(1)$
 - $A(-1) = A_{\text{even}}(1) - 1 A_{\text{odd}}(1)$
 - $A(i) = A_{\text{even}}(-1) + i A_{\text{odd}}(-1)$
 - $A(-i) = A_{\text{even}}(-1) - i A_{\text{odd}}(-1)$

can evaluate polynomial of degree $n-1$ at 4 points by evaluating 2 polynomials of degree $\frac{1}{2} n - 1$ at 2 points

can evaluate polynomial of degree $n-1$ at n points by evaluating 2 polynomials of degree $\frac{1}{2} n - 1$ at $\frac{1}{2} n$ points

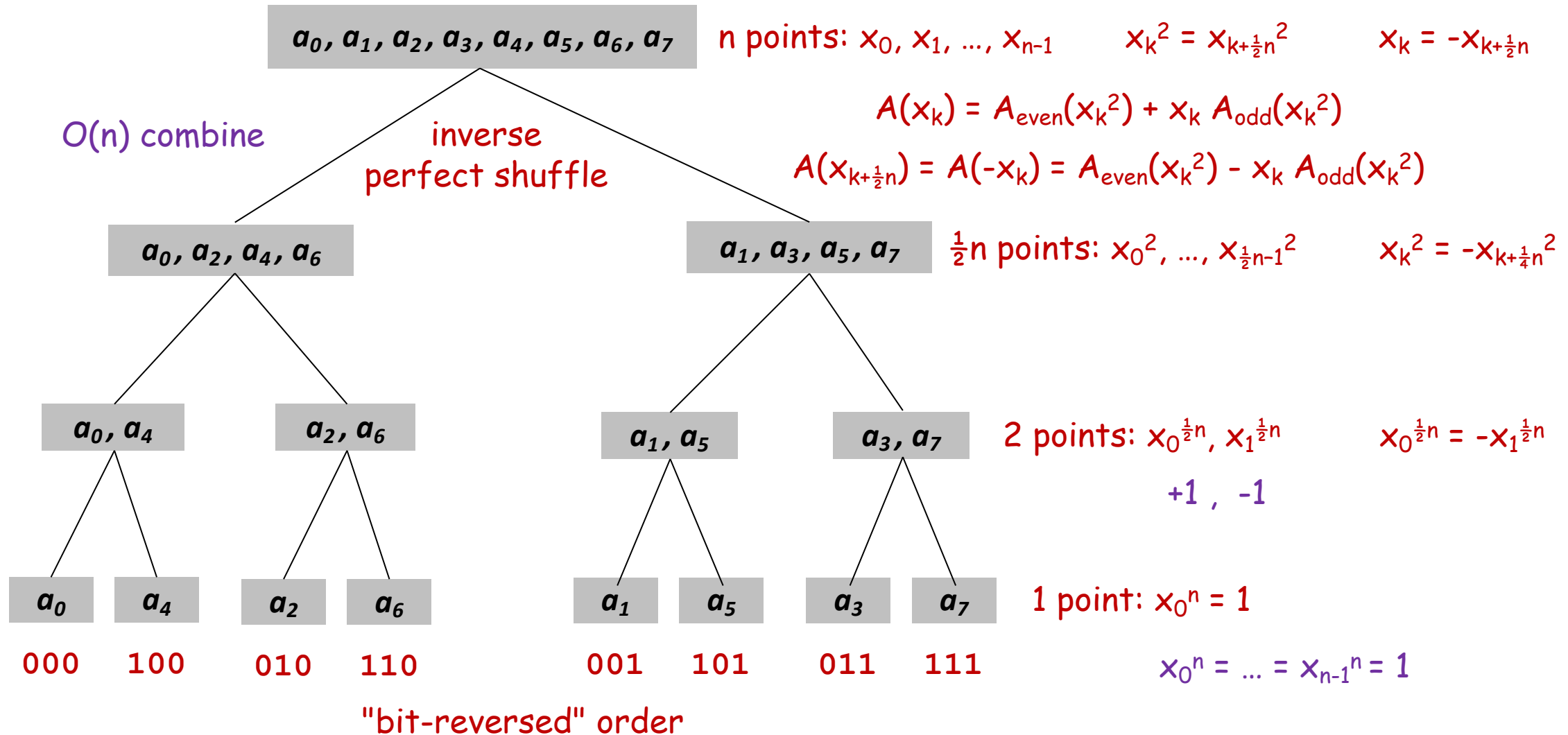


Coefficient to Point-Value: Recursion Tree





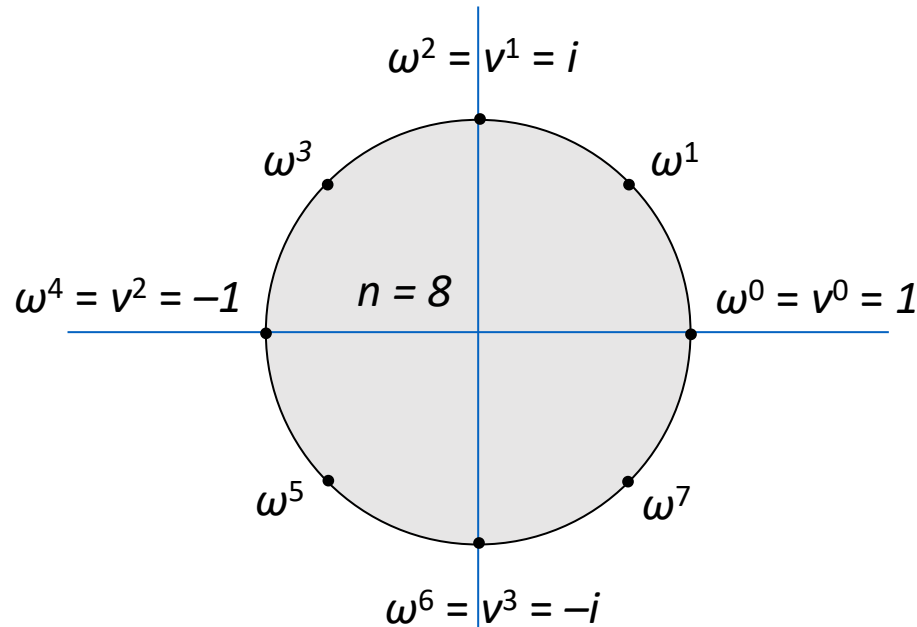
Coefficient to Point-Value: Recursion Tree





Roots of Unity

- **Def.** An n^{th} root of unity is a complex number x such that $x^n = 1$.
- **Fact.** The n^{th} roots of unity are: $\omega^0, \omega^1, \dots, \omega^{n-1}$ where $\omega = e^{2\pi i / n}$.
- **Pf.** $(\omega^k)^n = (e^{2\pi i k / n})^n = (e^{\pi i})^{2k} = (-1)^{2k} = 1$ common alternative: $\omega = e^{-2\pi i / n}$
- **Fact.** The $\frac{1}{2}n^{\text{th}}$ roots of unity are: $v^0, v^1, \dots, v^{\frac{1}{2}n-1}$ where $v = \omega^2 = e^{4\pi i / n}$.





Discrete Fourier Transform (DFT)

- **Coefficient $\Rightarrow$ Point-Value.** Given a polynomial $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$. $\leftarrow$ now let's choose these points!
- **Key idea:** choose $x_k = \omega^k$ where $\omega = e^{2\pi i/n}$ is principal n^{th} root of unity.

$$y_k = A(\omega^k) \rightarrow \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

$\uparrow$ DFT $\uparrow$ Fourier Matrix F_n



Fast Fourier Transform (FFT)

- **Goal.** Evaluate a degree $n - 1$ polynomial $A(x) = a_0 + \dots + a_{n-1} x^{n-1}$ at its all n^{th} roots of unity: $\omega^0, \omega^1, \dots, \omega^{n-1}$.
- **Divide.** Break up polynomial into **even** and **odd** powers.
 - $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \dots + a_{n-2} x^{\frac{1}{2}n-1}$
 - $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \dots + a_{n-1} x^{\frac{1}{2}n-1}$
 - $A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2)$
 - $A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2)$
- **Conquer.** Evaluate $A_{\text{even}}(x), A_{\text{odd}}(x)$ at $\frac{1}{2}n^{\text{th}}$ roots of unity: $v^0, v^1, \dots, v^{\frac{1}{2}n-1}$
- **Combine.** (Note that $v = \omega^2, v^{\frac{1}{2}n} = 1, \omega^{\frac{1}{2}n} = -1$ and $-\omega^k = \omega^{k+\frac{1}{2}n}$.)
 - $A(\omega^k) = A_{\text{even}}(v^k) + \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$
 - $A(-\omega^k) = A_{\text{even}}(v^k) - \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$



Fast Fourier Transform (FFT): Algorithm

- (Top-Down) Divide-and-Conquer algorithm:

```
FFT(n, a0, a1, ..., an-1) {  
    if (n = 1) return a0  
    (e0, e1, ..., en/2-1) = FFT(n/2, a0, a2, a4, ..., an-2)  
    (d0, d1, ..., dn/2-1) = FFT(n/2, a1, a3, a5, ..., an-1)  
  
    for k = 0 to n/2 - 1 {  
        ωk = e2πik/n  
        yk      = ek + ωk dk  
        yk+n/2 = ek - ωk dk  
    }  
  
    return (y0, y1, ..., yn-1)  
}
```

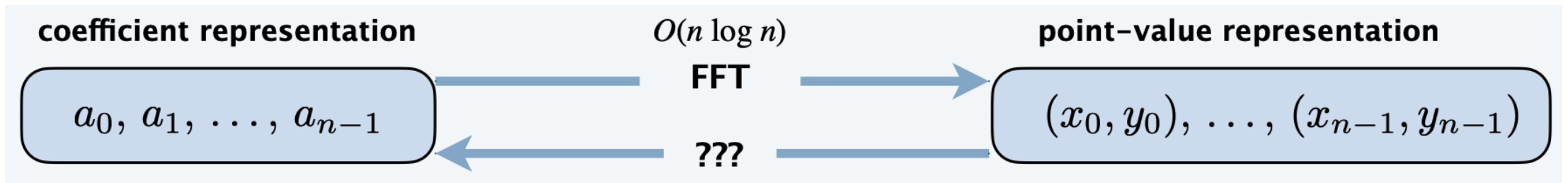
$2T(n/2)$

$O(n)$



FFT Summary

- **Theorem.** The **FFT** algorithm evaluates a degree $n - 1$ polynomial at each of the n^{th} roots of unity in $O(n \log n)$ operations. ↗ assume n is a power of 2
- **Pf.** $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$
- **Q.** What about space complexity?
- **A.** $O(n)$ ← bottom-up FFT implementation **Lab 7A**





Point-Value to Coefficient: Inverse DFT

- **Point-Value $\Rightarrow$ Coefficient.** Given n distinct points $x_0, \dots, x_{n-1}$ and values $y_0, \dots, y_{n-1}$, find unique polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ that has given values at given points.

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix}^{-1} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

↑
inverse DFT

↑
Fourier matrix inverse F_n^{-1}



Inverse Discrete Fourier Transform

- **Claim.** Inverse of Fourier matrix F_n is given by following formula:

$$G_n = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3} & \cdots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6} & \cdots & \omega^{-2(n-1)} \\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9} & \cdots & \omega^{-3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \omega^{-3(n-1)} & \cdots & \omega^{-(n-1)(n-1)} \end{bmatrix}$$

- **Consequence.** To compute the **inverse FFT**, apply the same algorithm but use $\omega^{-1} = e^{-2\pi i / n}$ as principal n^{th} root of unity (and **divide the result by n**).



Inverse FFT: Proof of Correctness

- **Claim.** F_n and G_n are inverses.

- **Pf.**

$$(F_n G_n)_{kk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{kj} \omega^{-jk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{(k-k')j} = \begin{cases} 1 & \text{if } k = k' \\ 0 & \text{otherwise} \end{cases}$$

summation lemma (below)

- **Summation Lemma.** Let ω be a principal n^{th} root of unity. Then

$$\sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n & \text{if } k \equiv 0 \pmod{n} \\ 0 & \text{otherwise} \end{cases}$$

- **Pf. (direct proof)**

- If k is a multiple of n then $\omega^k = 1$, so the series sums to n .
- Each n^{th} root of unity ω^k is a root of $x^n - 1 = (x - 1)(1 + x + x^2 + \dots + x^{n-1})$.
- If $\omega^k \neq 1$, then $1 + \omega^k + \omega^{2k} + \dots + \omega^{(n-1)k} = 0$, so the series sums to 0 . ■



Inverse FFT: Algorithm

- (Top-Down) Divide-and-Conquer algorithm:

```
Inverse-FFT( $n, y_0, y_1, \dots, y_{n-1}$ ) {  
    if ( $n == 1$ ) return  $a_0$   
    ( $e_0, e_1, \dots, e_{n/2-1}$ ) = Inverse-FFT( $n/2, y_0, y_2, y_4, \dots, y_{n-2}$ )  
    ( $d_0, d_1, \dots, d_{n/2-1}$ ) = Inverse-FFT( $n/2, y_1, y_3, y_5, \dots, y_{n-1}$ )  
  
    for  $k = 0$  to  $n/2 - 1$  {  
         $\omega^k = e^{-2\pi i k / n}$   
         $a_k = e_k + \omega^k d_k$   
         $a_{k+n/2} = e_k - \omega^k d_k$   
    }  
    return ( $a_0, a_1, \dots, a_{n-1}$ )  
}
```

$2T(n/2)$

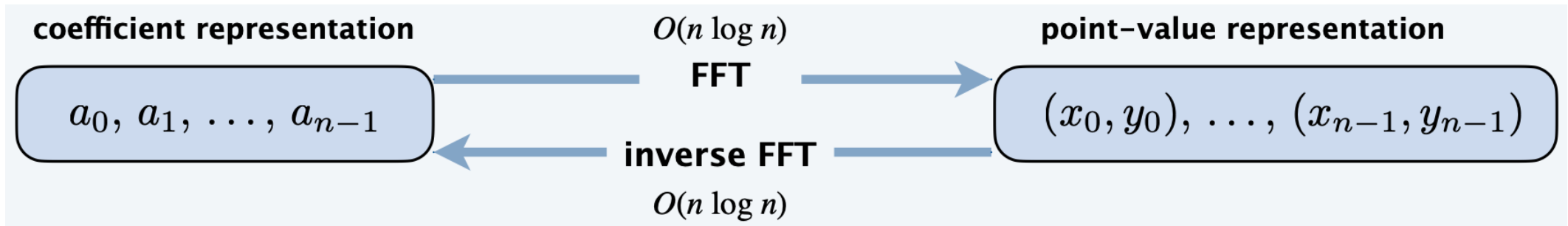
$O(n)$

Output: $\text{Inverse-FFT}(n, y_0, y_1, \dots, y_{n-1}) / n$



Inverse FFT Summary

- **Theorem.** The **inverse FFT** algorithm interpolates a degree $n - 1$ polynomial at each of the n^{th} roots of unity in $O(n \log n)$ operations.
↖ assume n is a power of 2
- **FFT + Inverse FFT.** Can convert between coefficient and point-value representations in $O(n \log n)$ arithmetic operations.

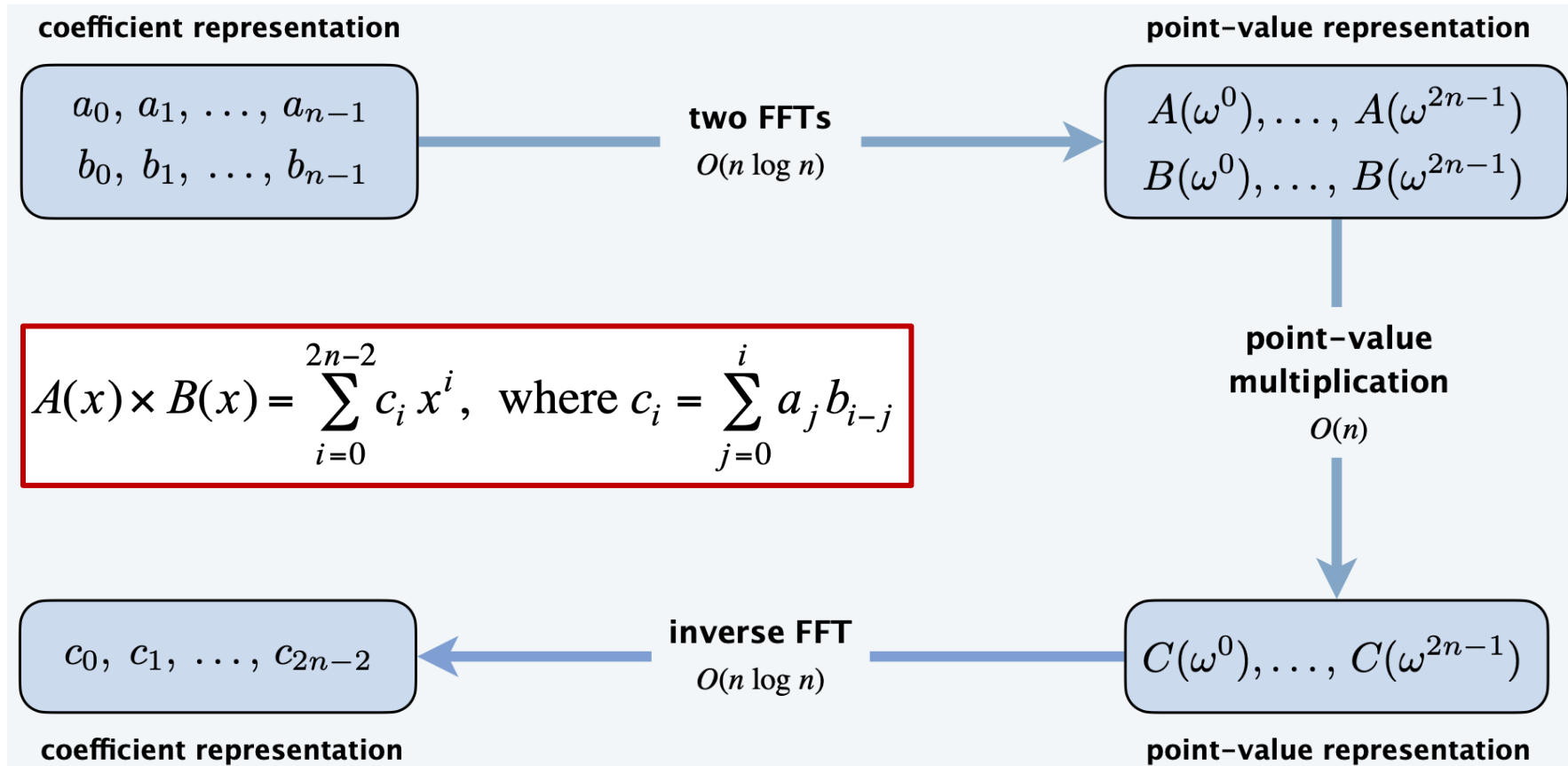




Fast Polynomial Multiplication

- **Theorem.** Can multiply two degree $n - 1$ polynomials in $O(n \log n)$ steps.
- **Pf.**

pad 0 items to make n a power of 2





Integer Multiplication Revisited

- **Integer Multiplication.** Given two n bit integers $a = a_{n-1} \dots a_1 a_0$ and $b = b_{n-1} \dots b_1 b_0$, compute their product $c = ab$.
- **Convolution algorithm:**
 - Form two polynomials $A(x), B(x)$.
$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$$
 - Note that $a = A(2), b = B(2)$.
$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$$
 - Compute $C(x) = A(x) B(x)$ and evaluate $C(2) = ab$.
 - Running time: $O(n \log n)$ floating-point operations ($O(n \log^3 n)$ bit operations).
- **Optimization:** [Schönhage-Strassen 1971]
 - $O(n \log^2 n)$ bit operations over complex numbers (with $O(\log n)$ bit precision)
 - $O(n \log n \log \log n)$ bit operations over ring of integers (mod a Fermat number)
- **Practice.** GNU Multiple Precision Arithmetic Library (GMP) switches to FFT-based algorithms when n is large ($\geq \approx 5000$).



FFT in Practice

- **FFT in the West (FFTW): [Frigo-Johnson]**

- Optimized C library.
- Features: DFT, DCT, real, complex, any size, any dimension.
- Won 1999 Wilkinson Prize for Numerical Software.
- Portable, competitive with vendor-tuned code.

- **Implementation details:**

- Core algorithm is an in-place, non-recursive version of Cooley-Tukey.
- Instead of executing a fixed algorithm, it evaluates the hardware and uses a special-purpose compiler to generate an optimized algorithm catered to the “shape” of the problem.
- Runs in $O(n \log n)$ time, even when n is prime.
- Multidimensional FFTs.
- Parallelism.

The FFTW logo is located in the bottom-right area of the slide. It consists of the letters "FFTW" in a bold, sans-serif font. The "FF" and "T" are black, while the "W" is a vibrant red.

Reference: <http://www.fftw.org>



Announcements

- **Assignment 3 has been released and the deadline is Apr 22.**
 - In this assignment, you will be asked to investigate other FFT applications, and/or FFT variants and their applications.
- **Lab 7 will be released today and the deadline is Apr 22.**